Stream A — OpenDesign Integration + Smooth Transitions + Look-and-Feel Polish (design)

Date: 2026-06-26 **Status:** DRAFT — operator decisions on 6 questions required before Wave A execution (see §6) **Operator directive (2026-06-25):** "Make transitions between all application's screens, fragments and views more smooth with some stunning visual effects! Do use heavily OpenDesign and fine tune and polish all existing look and feel! Cover all with additional tests!!!" **Relationship to specs:** Standalone UI/UX stream. Complements the search/provider-resolution functional bugfix track (separate operator directive); the video's CRITICAL functional items (#1 zero-results, #2/#3 provider-set mismatch) are OUT of scope and belong to that track. **Constitution anchor:** §11.4.162 (OpenDesign UI design-system mandate) — inherited via §6.AD from `constitution/`. **Anti-bluff classification:** `Classification:` universal (OpenDesign compliance + visual-regression + transition discipline pattern is reusable across any Helix-constitution-consuming Android project; the specific 5-wave plan, file paths, and test names are project-specific).

---

1. Current state audit

All claims below cite the specific files and line ranges read in the scoping analysis (`docs/qa/2026-06-25-opendesign-transitions-scoping.md`). No code was changed — this is a read-only audit.

1.1 OpenDesign compliance (§11.4.162) — NOT integrated

Finding: OpenDesign is absent from Lava. A grep across `gradle/libs.versions.toml`, every `build.gradle.kts`, and `core/designsystem/` for `open-design` / `opendesign` / `nexu` returns zero hits in Lava's own source. The only matches are sibling project copies (`panoptic/CLAUDE.md`, `panoptic/AGENTS.md`, `panoptic/CONSTITUTION.md` — not Lava). Lava is §11.4.162 non-compliant.

What OpenDesign actually is:

- OpenDesign (`github.com/nexu-io/open-design`, v0.7.0) is a **local-first, open-source design-systems tool**: a native desktop app + stdio MCP server + per-agent install scripts, shipping 142+ design systems.
- A "design system" in OpenDesign is a folder of: `manifest.json`, `DESIGN.md` (canonical design prose), `tokens.css` (compiled CSS custom properties — the canonical token artifact), `components.html`, `assets/`, `fonts/`, preview pages.
- **It is NOT a Maven/Gradle artifact and NOT a Kotlin/Compose library.** There is no `androidx`-style dependency to add to `libs.versions.toml`.
- "Use OpenDesign" in the Android context means: (1) install the MCP server (`od mcp install claude-code`), (2) author/adopt a Lava-branded design system with `tokens.css` as the canonical token artifact, (3) generate/sync the Compose token layer (`AppColors`, `AppTypography`, `AppSpaces`, `AppShapes`) **from** that `tokens.css`.

1.2 Theming — mature but hand-coded, no brand font, no visual-regression tests

`core/designsystem/` (`LavaTheme`):

- **Theme entry:** `theme/Theme.kt:14-72` — `LavaTheme(theme, isDark, isDynamic, content)`. Maps a `Theme` enum to a palette factory, builds Material3 `lightColorScheme`/`darkColorScheme`, provides `LocalColors`.
- **Dark theme: YES, comprehensive.** Every palette factory takes `isDark` and returns full dark+light variants; `isSystemInDarkTheme()` is the default (`:17,27-37`).
- **8 named palettes + Material You:** `yoleColors`, `draculaColors`, `solarizedColors`, `nordColors`, `monokaiColors`, `gruvboxColors`, `oneDarkColors`, `tokyoNightColors`, plus `DYNAMIC` (API 31+ via `isMaterialYouAvailable()`). (`:21-37,74-75`)
- **Brand colors:** hand-coded Material tonal ramps in `theme/Colors.kt:1-180` — ~180 lines of literal `Color(0xFF…)` constants. Note: the "Indigo*" ramp is the brand red (`Indigo40 = 0xFFB3261E`, `Indigo50 = 0xFFDE3730`); "Studio*" = purple secondary; "Lipstick*" = pink tertiary.
- **Token classes already exist:** `AppColors`, `AppTypography`, `AppShapes`, `AppSizes`, `AppSpaces`, `AppBorders`, `AppElevations` in `theme/AppTheme.kt:6-41`.
- **Typography:** full Material type scale — **but every style uses `fontFamily = FontFamily.Default`** (`AppTypography.kt:11-117`). No brand font. No `.ttf`/`.otf` assets in Lava.
- **Components:** ~25 design-system composables under `component/` (AppBar, Buttons, Dialog, TextField, NavigationBar, ModalBottomSheet, Placeholder, ProgressIndicator, Scaffold, Surface, Pagination, etc.).

- **Theming tests:** `PaletteContractTest.kt` (Robolectric) — asserts 8 palettes produce valid light+dark colors, `primary != surface`. `LavaIconsAppIconColorRegressionTest`, `A11yContentDescriptionTest`. **No visual-regression / screenshot infra** (no Paparazzi, no Roborazzi, no perceptual-diff — grep confirms none). This is the §11.4.162 "visual regression tests with per-pixel/perceptual diff" gap.

1.3 Transitions — a system exists, but most destinations use `Default` (= no transition)

The transition machinery lives in `core/navigation` and is wired per-destination in `:app`:

- **Animation model:** `navigation/ui/NavigationAnimations.kt` — `NavigationAnimations(enter/exit/popEnter/popExit)` data class with presets.
  - `Default` = all-null = **NavHost default (no custom transition)** (`:29`).
  - `ScaleInOutAnimation` = scaleIn+expandIn+fadeIn / fadeOut+shrinkOut+scaleOut (`:30-49`).
  - `FadeInOutAnimations` = fadeIn / fadeOut (`:50-55`).
  - `slideInLeft/slideOutLeft/slideInRight/slideOutRight` helpers (`:57-60`).
- **Wiring:** `NavigationHost.kt:42-56` plumbs these into Navigation-Compose.
- **Per-destination assignment** (`app/.../navigation/MobileNavigation.kt`): | Destination | Current animation | Status | |---|---|---| | login, credentials, credentialsManager, providerConfig | `ScaleInOutAnimation` (`:55-68`) | OK | | category, topic | `ScaleInOutAnimation` (`:97,105`) | OK | | searchInput (top-level), searchResult (top-level) | **`Default` (none)** (`:76,88`) | **GAP — abrupt cut** | | bottom-nav tabs | directional `slideIn/Out` by tab ordinal (`:329-350`) | OK | | nested search graph: searchHistory `Default`, searchInput `FadeInOutAnimations`, searchResult `Default` | mixed (`:178,186,194`) | PARTIAL |
- **In-screen animations that DO exist:**
  - Onboarding: `AnimatedContent` with 320ms `FastOutSlowInEasing` slide + 220ms fade (`feature/onboarding/.../OnboardingScreen.kt:109-132`) — the current best-in-class.
  - Topic: `fadeIn() + slideInVertically { it }` on a sub-element (`feature/topic/.../TopicScreen.kt:393`).
- **What's ABSENT entirely:**
  - **No shared-element / shared-bounds transitions** anywhere (grep confirms zero `SharedTransitionLayout` / `sharedElement` usage). Tapping a search-result row → topic does a scale, not a morph.
  - **The two highest-traffic transitions (search-input → results, results → topic at the top level) are `Default` = no animation** — abrupt cuts.

    - No global/consistent transition spec — easing/duration is ad-hoc per call site.

1.4 UI-coverage audit (from `docs/qa/2026-06-25-ui-coverage-audit.md`)

- ~45% of screens have a behavior-asserting UI test (Challenge)
- ~25% WEAK (reachable-only via `Class.forName`, no behavior assertion)
- ~30% GAP (no UI test at all)
- **0%** visual-regression (no golden-image/perceptual-diff infra)
- This spec's Wave D + Wave E close these gaps.

---

2. Problem

Lava's UI is §11.4.162 non-compliant — OpenDesign is not integrated, visual-regression tests do not exist, and transition quality is inconsistent. End users experience:

1. **Abrupt screen transitions on high-traffic paths.** The search-input → results and results → topic top-level destinations use `Default` (= no animation), producing cuts that feel unpolished. Compare with the polished onboarding (320ms slide+fade) — the gap is jarring.
2. **No shared-element transitions anywhere.** Tapping a search-result row replays a generic scale animation instead of morphing the poster/title to the topic detail. This is the single biggest "stunning visual" gap.
3. **Missing loading/empty states on search results.** The #1 visual defect from the operator's video: search shows a blank/black body for ~25 s then jumps to a full-screen error. No skeleton, no spinner, no "No results" state.
4. **Raw provider IDs shown instead of display labels.** Chips render "torrentdownloads", "archiveorg", "kinozal" instead of human-readable names — same class as the §6.L 60th `displayLabel()` underscore bug.
5. **mDNS-discovered API shows bare IP:port** with no friendly name.
6. **"4 providers available" copy is wrong** vs ~12 actual.
7. **Inconsistent token application** across screens — `AppSpaces`/`AppTypography` are applied ad-hoc, not enforced by a single source-of-truth.
8. **No brand font** — all `FontFamily.Default`.
9. **No visual-regression layer** — a theming/transition/overlap regression is invisible to the test suite.

Each of these is independently user-visible, independently testable via device Challenge, and independently fixable. The spec addresses all of them across 5 waves.

---

3. Goal / non-goals

Goal

Make Lava's look-and-feel **§11.4.162 compliant**: OpenDesign-sourced design tokens, smooth and tasteful screen transitions including shared-element morphs, consistent spacing/typography/color across all surfaces, a brand font, and a visual-regression test layer that prevents regressions. Every UI change is covered by a falsifiable covering device Challenge.

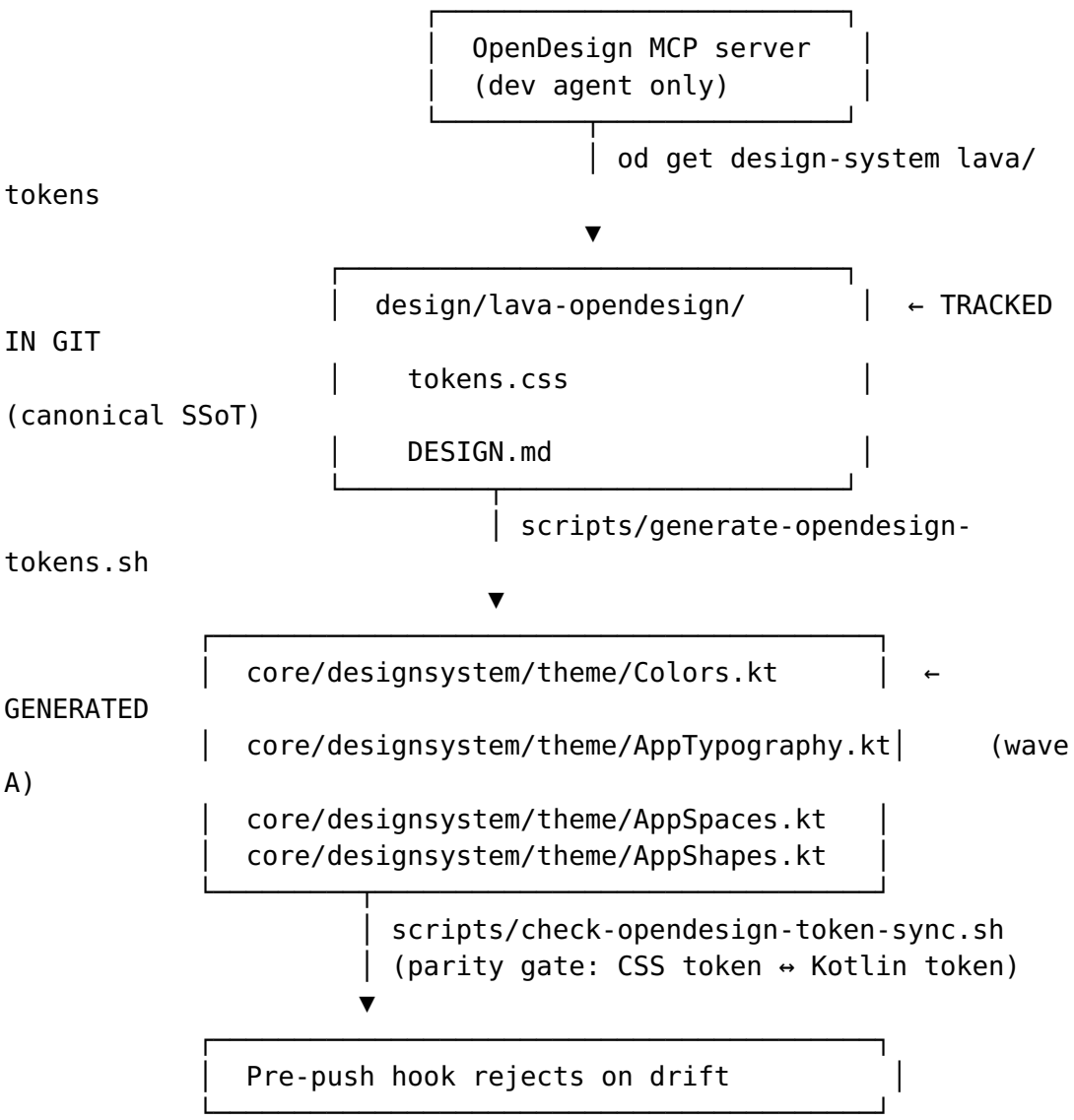
Non-goals (explicitly, per anti-bluff §6.J honesty)

1. **NOT a general UI rewrite.** The existing `AppColors`/`AppTypography`/`AppSpaces` architecture is kept; only its *source* changes (from hand-coded literals to OpenDesign-generated). No composable replacement, no screen redesign.
2. **NOT Google's Material Motion library adoption.** Lava uses Navigation-Compose's `enterTransition`/`exitTransition` API augmented with `SharedTransitionLayout` (Compose Foundation) — no extra dependency.
3. **NOT fixing functional bugs** (#1 zero-results, #2/#3 provider-set mismatch from the video). Those are the separate search/provider-resolution track. This spec owns *visual/transition* defects (#4 loading/empty states, #5 raw IDs, #8 IP display, #6 provider count) + the global polish.
4. **NOT a full Roborazzi/Paparazzi campaign across every screen.** Wave D targets design-system components + key screens (the ~45% that have behavior tests today); Wave E then extends coverage to the 30% GAP. The visual-regression layer grows incrementally.
5. **NOT a runtime dependency on OpenDesign.** `tokens.css` is committed to the repo as the SSoT; the Compose codegen produces Kotlin token classes from it. The OpenDesign MCP server is a *development-time* tool (agent-side), not a runtime dependency on users' devices.
6. **NOT fully on-device executable on darwin/arm64.** Rendered-UI Challenges on real emulators remain gated by the standing §6.AH/§6.X-debt gap. Visual-regression tests (Roborazzi/Paparazzi) run host-side JVM. All new Challenges document their gating status in the KDoc.

---

4. Architecture

4.1 Token pipeline: `tokens.css` → Compose codegen → parity gate

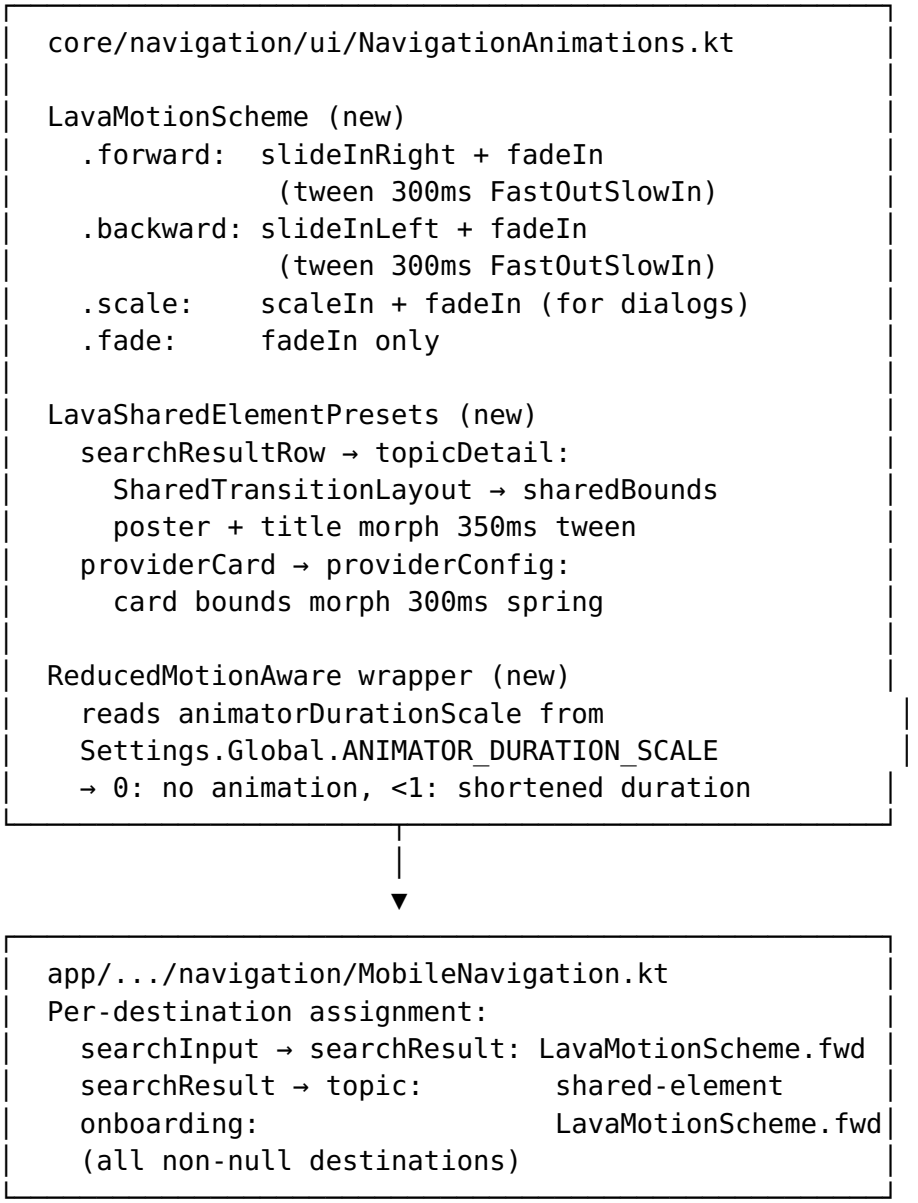


Key properties:

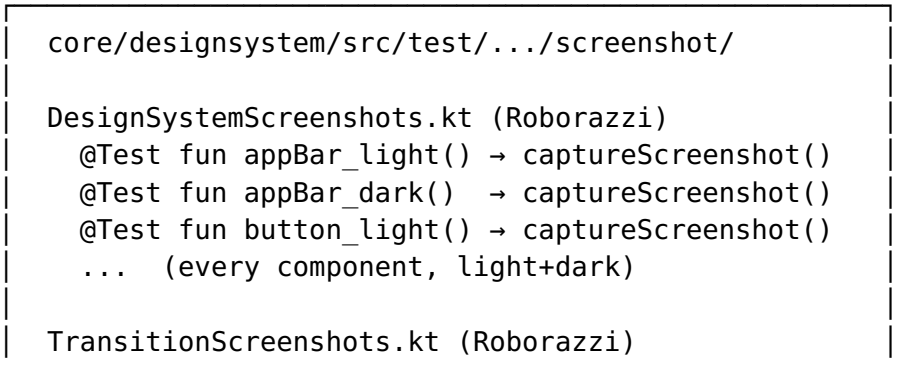
- `tokens.css` is the **single source of truth** for all design tokens. Hand-coded Kotlin color/type/space/shape literals are replaced by codegen output.
- The codegen script (`scripts/generate-opendesign-tokens.sh`) is a thin block of Kotlin script or Python that parses CSS custom properties and emits Kotlin source. It runs on demand (after `tokens.css` changes) and is checked into git alongside its output.
- The parity gate (`scripts/check-opendesign-token-sync.sh`) verifies that the committed Kotlin tokens match the CSS tokens exactly — fails on drift. This is the §6.A real-binary contract discipline applied to design tokens.

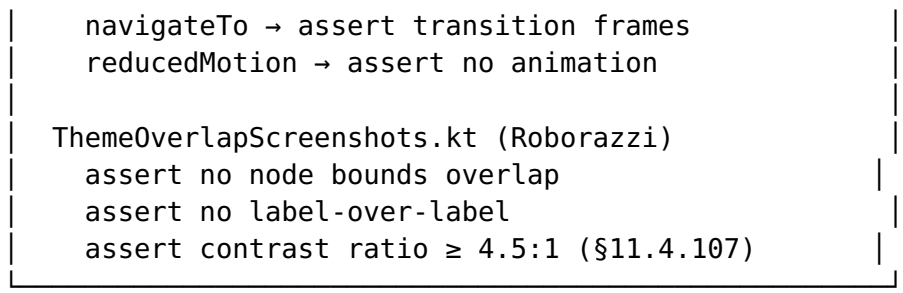
- **No runtime dependency on OpenDesign.** The MCP server is only used by the agent during development.

4.2 Transition architecture



4.3 Visual-regression architecture (Wave D)





Roborazzi is the recommended tool (reuses existing Compose test wiring). Both Roborazzi and Paparazzi are JVM-native (Robolectric-backed), so this layer runs host-side without an emulator. See §6 decision #5 for operator confirmation.

---

5. §6.AK compliance — per-wave device Challenge requirements

Per §6.AK (Cycle-Coverage Device Gate, added 2026-06-26), every user-visible fix claimed in the CHANGELOG MUST have a covering device Challenge that (a) was reproduce-first proven (RED on the unfixed build, GREEN on the fixed build), and (b) was EXECUTED on the §6.Z device gate against the EXACT artifact being shipped.

The table below defines per-wave Challenge requirements. These are **minimum requirements** — additional Challenges may be added during implementation. Each Challenge's KDoc MUST include a `FALSIFIABILITY REHEARSAL` block naming the deliberate mutation and the expected failure message.

Wave	Claimed user-visible fix	Covering Challenge	Mutation f run
A	OpenDesign tokens applied, visual tokens match CSS SSoT	`scripts/check-opendesign-token-sync.sh` parity gate (hermetic test)	Remove a c `tokens.css`, gate → EXI report
B	searchInput → searchResult has smooth slide+fade transition	C-XX (new) `ChallengeSearchInputToResultsTransitionTest`	Replace `LavaMotionS` with `Defaul` `MobileNaviga` run → asser `enterTransi` for the top-

Wave	Claimed user-visible fix	Covering Challenge	Mutation f run
			searchResu FAILS
B	searchResult → topic shared-element morph of poster/title	C-XX (new) `ChallengeSearchResultToTopicSharedElementTest`	Comment o `sharedEleme` `sharedBounds` topic compo run → golde visual diff d missing mo FAILS
B	Reduced-motion mode disables animations	C-XX (new) `ChallengeReducedMotionTransitionTest`	Override `animatorDura` `1` when `0` is re-run → tra frames are FAILS (expe frames)
C	Search results show Loading skeleton + Empty state (video #5)	C-XX (new) `ChallengeSearchResultLoadingEmptyTest`	Remove the `(loading)` br `SearchResult` run → skele composable FAILS
C	Provider chips show display labels, not IDs (video #4)	C-XX (new) `ChallengeProviderChipDisplayLabelTest`	Revert `disp` `name.lowerca` → chip text raw id "torrentdov FAILS
C	mDNS API shows friendly name, not bare IP (video #8)	C-XX (new) `ChallengeMdnsFriendlyNameTest`	Return `""` fr name resolv screen show IP:port, FAI
C	Provider count copy is correct (video #6)	C-XX (new) `ChallengeProviderCountLabelTest`	Hardcode c re-run → lab providers a FAILS on w
C	Brand font renders in typography	C-XX (new) `ChallengeBrandFontAppliedTest`	Revert to `FontFamily.D` run → visua golden dete font metrics
D	Visual-regression	Existing C-XX design-system golden tests	Flip a color `Colors.kt` (e

Wave	Claimed user-visible fix	Covering Challenge	Mutation f run
	golden tests catch rendering break		`= Color.Red` golden perc > threshold
D	Transition tests assert animation presence	Transition-specific golden tests	Remove `ent` assignment assertion or transition,
D	Contrast/ overlap tests detect overlap	Theme-overlap test	Add overlap composable overlap ass
E	Rewritten C31-C35 behave correctly	New C31-C35 with behavior + visual assertions	Per-Challen in KDoc fals block

Per-wave `git diff` validation: Before each wave's coverage is accepted, the implementer MUST confirm that every file touched in the wave's commits is covered by at least one of the wave's Challenges. Uncovered file modifications are a §6.AK violation.

---

6. Decision framework — operator input needed before Wave A execution

Six decisions are blocking Wave A. Each is formulated as a question with a recommendation.

Decision 1: Author vs adopt design system

Question: Should Lava author a bespoke OpenDesign design system from the brand palette (`0xFFB3261E` red), or adopt+rebrand one of the 142 shipped systems? **Recommendation: Author a bespoke Lava system** (`design/lava-opendesign/`) that extends a minimal base (e.g., Material 3). Per §11.4.74 extend-don't-reimplement: authoring a Lava-specific system is the correct choice because Lava's brand palette (red secondary, purple tertiary, pink accent) is distinctive, and the ~25 existing composables encode Lava-specific layouts that a generic system would not match. **Operator needs to:** Confirm the brand palette is correct (canonical hex values for the 3 brand colors). Provide any brand color guidance beyond what `Colors.kt` already encodes.

Decision 2: Brand font

Question: Lava has no brand typography — every style is `FontFamily.Default`. Which brand font should Lava use? **Recommendation: Use a libre/open-source font** to avoid licensing friction. Recommended candidates: **Inter** (sans-serif, excellent legibility, variable-weight support, OFL license) or **Manrope** (modern geometric sans, OFL). Both have variable `.ttf` that works on Android. Inter is the safer choice (broader character support, extensive weight range). **Operator needs to:** Choose a font (or confirm "Inter") and provide a `.ttf`/`.otf` file path or source URL. The font asset is placed at `core/designsystem/src/main/assets/fonts/` and wired in `AppTypography.kt`.

Decision 3: `tokens.css` as tracked SSoT

Question: Should `tokens.css` be committed to the repo as the canonical token SSoT, with the parity gate as the contract? **Recommendation: Yes.** This is the standard pattern for OpenDesign integration in non-web projects. Committing `tokens.css` makes it versioned, diffable, and accessible to all agents regardless of MCP server availability. The parity gate (`check-opendesign-token-sync.sh`) prevents drift. **Operator needs to:** Confirm. No further action.

Decision 4: `od` CLI availability

Question: Is the OpenDesign CLI (`od`) available on the gate host? If not, who installs it? **Recommendation:** OpenDesign 0.7.0 is open-source/local-first, so no paid account is needed. The installation is a one-liner per agent (`od mcp install claude-code`). The `tokens.css` can also be authored/edited manually without the `od` CLI — the MCP server is a development convenience, not a gate dependency. **Operator needs to:** Confirm that `od` is available on the primary development machine. The parity gate runs regardless of `od` presence (it compares committed files).

Decision 5: Visual-regression tool — Roborazzi vs Paparazzi

Question: Which JVM-compatible visual-regression library should Lava adopt? Both run on Robolectric (no device needed), both fit Local-Only CI/CD. **Recommendation: Roborazzi.** It integrates directly with Compose UI tests (reuses the existing `createComposeRule()` pattern in `core:testing`), supports `captureScreenshot()` as a composable test assertion, and has an active maintenance cadence. Paparazzi is render-only (no interaction) and would require a separate test harness. **Operator**

needs to: Confirm. If Roborazzi is chosen, the implementer will add `robolectric` + `roborazzi` dependencies to `libs.versions.toml` and `core/designsystem/build.gradle.kts`.

Decision 6: Scope boundary — visual vs functional fixes

Question: The operator's video contains both visual defects (this cycle's ownership) and functional bugs (#1 zero-results, #2/#3 provider-set mismatch). Should the functional bugs block Wave C (which touches `search_result` and provider-related screens)? **Recommendation: No — keep them separate tracks.** Wave C's `feature/search_result` changes (Loading/Empty states, chip labels) are new composables and label mappings. They do not fix the search-zero-results functional bug, but they *improve the visual experience regardless* (a blank screen becomes a skeleton; raw IDs become display names). The functional search fix track is a separate operator directive and lands independently. **Operator needs to:** Confirm this split. If Wave C should additionally fix the functional search bug, the scope expands and this spec's timeline changes.

---

7. 5-Wave implementation plan

Each wave is scoped to disjoint files where possible so waves can run as parallel agents. Every commit MUST land light+dark variants and carry a §6.AB.3 falsifiability rehearsal in the commit body. Dependencies: A → B (A's transition spec informs B's easing), A → C (C consumes A's tokens). D and E parallelize after A.

Wave A — OpenDesign integration + token source-of-truth (FOUNDATION)

Depends on: Operator decisions 1–4 (design system, brand font, `tokens.css`, `od` CLI)

Files created:

- `design/lava-opendesign/tokens.css` — canonical CSS custom properties (light+dark palette, type scale, spacing, shapes, component tokens)
- `design/lava-opendesign/DESIGN.md` — design narrative per OpenDesign convention
- `design/lava-opendesign/manifest.json` — OpenDesign manifest
- `design/lava-opendesign/components.html` — component listing (minimal, generated)

- `core/designsystem/src/main/assets/fonts/<brand-font>.ttf` — brand font asset
- `scripts/generate-opendesign-tokens.sh` — codegen: parses `tokens.css` → emits `Colors.kt.update`, `AppTypography.kt.update`, `AppSpaces.kt.update`, `AppShapes.kt.update`
- `scripts/check-opendesign-token-sync.sh` — parity gate: exits non-zero if CSS token ≠ Kotlin token
- `scripts/install-opendesign.sh` — one-shot MCP server installer for agents (`od mcp install claude-code`, etc.)
- `docs/opendesign-integration.md` — companion guide (mirroring `docs/CODEGRAPH.md` style)

Files modified:

- `core/designsystem/theme/Colors.kt` — generated section replaces hand-coded literals (hand-coded section preserved with `// region HandCraftedOverrides` for overrides)
- `core/designsystem/theme/AppTypography.kt` — `FontFamily.Default` → brand font, generated from `tokens.css`
- `core/designsystem/theme/AppSpaces.kt` — generated from `tokens.css`
- `core/designsystem/theme/AppShapes.kt` — generated from `tokens.css`
- `.lava-ci-evidence/verify-all/` — parity gate registration for sweep

Tests created:

- `tests/designsystem/test-token-parity.sh` — hermetic test of `check-opendesign-token-sync.sh` (positive: matching tokens → EXIT 0; negative: drift → EXIT 1; falsifiability: mutate CSS, confirm gate fails)

Operator action required before execution: Decisions 1–4.

Wave B — Shared screen-transition system (the "smooth + stunning" core)

Depends on: Wave A (for the motion design language informant). No code dependency — easing constants are independent of `tokens.css`.

Files created:

- `core/navigation/ui/LavaMotionScheme.kt` — `LavaMotionScheme` object with `forward`/`backward`/`scale`/`fade` specs (300ms tween, `FastOutSlowInEasing`)
- `core/navigation/ui/LavaSharedElementPresets.kt` — shared-element configs for searchResult→topic (poster+tile morph, 350ms) and providerCard→providerConfig (card morph, 300ms spring)

- `core/navigation/ui/ReducedMotionAwareTransition.kt` — wrapper that reads `ANIMATOR_DURATION_SCALE` and returns `null` (no animation) when scale == 0, shortened duration when scale < 1
- `core/designsystem/src/test/.../transition/LavaMotionSchemeTest.kt` — unit tests: asserts easing constants, duration ranges, scale order

Files modified:

- `app/.../navigation/MobileNavigation.kt` — replace `Default` assignments:
  - Line 76: `searchInput` (top-level) → `LavaMotionScheme.forward`
  - Line 88: `searchResult` (top-level) → `LavaMotionScheme.forward`
  - Line 178: nested `searchHistory` → `LavaMotionScheme.fade`
  - Line 186: nested `searchInput` → `LavaMotionScheme.forward`
  - Line 194: nested `searchResult` → `LavaMotionScheme.forward`
  - Lines 97, 105: `category`, `topic` → `LavaMotionScheme.scale` (keep existing ScaleInOut semantics but with tuned easing)
- `feature/search_result/.../SearchResultScreen.kt` — add `sharedBounds` on result row poster/title
- `feature/topic/.../TopicScreen.kt` — add `sharedElement` on poster/title to receive the morph
- `feature/search_input/.../SearchInputScreen.kt` — no change (it's the origin, kept clean)
- `feature/provider_config/.../ProviderConfigScreen.kt` — add `sharedElement` for card morph (receiving end)

New Challenges (device, gated per §6.AH):

- `C-XX-ChallengeSearchInputToResultsTransitionTest` — drives searchInput → searchResult; asserts `enterTransition` is not null for the result route
- `C-XX-ChallengeSearchResultToTopicSharedElementTest` — drives searchResult → topic; captures screenshot of transition mid-frame; asserts shared-element bounds match
- `C-XX-ChallengeReducedMotionTransitionTest` — sets `ANIMATOR_DURATION_SCALE` to 0; drives a nav destination; asserts no animation composables appear

Wave C — Per-feature polish via OpenDesign tokens

Depends on: Wave A (consumes generated `AppColors`/`AppTypography`/`AppSpaces`). Some items (chip labels, friendly name, provider count) are token-independent and can start before Wave A.

Files modified:

- `feature/search_result/.../SearchResultScreen.kt` — add Loading skeleton composable (when `state.isLoading`) + Empty state

composable (when `state.results.isEmpty() && ! state.isLoading`); wire to `AppTheme.colors/typography/spaces`
- `feature/search_result/.../SearchResultViewModel.kt` — ensure `state.isLoading` is exposed for the skeleton; ensure empty-state distinction (no results vs error)
- `core/designsystem/component/LoadingSkeleton.kt` — new composable: shimmer/skeleton placeholder matching OpenDesign token specs
- `core/designsystem/component/EmptyState.kt` — new composable: icon + message + optional action button, OpenDesign-token-driven
- `feature/search_input/.../SearchInputProviderChip.kt` — route chip label through `displayLabel()`/`displayName()` from the provider descriptor (fix video #4)
- `core/data/.../LocalNetworkDiscoveryService.kt` or consuming screen — add friendly-name resolution (prefer mDNS service name over bare IP; label as "Remote" vs "Local network") (fix video #8)
- `feature/onboarding/.../ProviderCountSection.kt` — fix `"4 providers available"` to count dynamically from `availableProviders.size` (fix video #6)
- `core/designsystem/theme/AppTypography.kt` — brand font already wired from Wave A

Tests created:

- `tests/designsystem/test-loading-skeleton-contract.kt` — JVM test: assert skeleton renders in both themes, has proper colors, does not overlap
- `tests/designsystem/test-empty-state-contract.kt` — JVM test: assert empty state renders icon + message, theme colors correct

New Challenges (device, gated per §6.AH):

- `C-XX-ChallengeSearchResultLoadingEmptyTest` — search with a query that returns no results; assert Empty state visible; assert skeleton appears during loading
- `C-XX-ChallengeProviderChipDisplayLabelTest` — open search input; assert every provider chip's text does not match its raw `tracker_descriptor.id` (i.e., it displays label, not id)
- `C-XX-ChallengeMdnsFriendlyNameTest` — with a mock/known mDNS API response; assert the friendly service name is shown, not the bare IP:port (requires mock mDNS or the `TestLocalNetworkDiscoveryService`)
- `C-XX-ChallengeProviderCountLabelTest` — navigate to onboarding → provider discovery; assert the count label reads `N providers available` where N > 4 (the old hardcoded value)

Wave D — Visual-regression + transition + theming test infrastructure

Depends on: Decision 5 (Roborazzi vs Paparazzi). Parallelizes with B and C.

Files created:

- `core/designsystem/src/test/.../screenshot/DesignSystemScreenshots.kt` — golden-screenshot test for every design-system composable (~25), light+dark
- `core/designsystem/src/test/.../screenshot/TransitionScreenshots.kt` — assert navigation transition frames via Roborazzi `captureTransitionScreenshot()` (if available) or custom `Animatable` snapshot
- `core/designsystem/src/test/.../screenshot/ThemeOverlapScreenshots.kt` — assert no node bounds overlap, no label-over-label, contrast >= 4.5:1 (§11.4.107)
- `core/designsystem/src/test/.../screenshot/PaletteOpenDesignParityTest.kt` — assert every palette's token values match the CSS declarations
- `core/designsystem/src/test/.../screenshot/ReducedMotionTest.kt` — JVM test: set `isEnabled = false` on the motion scheme, assert transition specs resolve to null

Files modified:

- `scripts/ci.sh` — register the visual-regression suite (`./gradlew :core:designsystem:testDebugUnitTest` with Roborazzi tasks)
- `scripts/check-opendesign-token-sync.sh` — extended to also verify golden images are fresh (compare golden file timestamps against source file timestamps)

Golden image management:

- Goldens live at `core/designsystem/src/test/.../screenshot/goldens/` (light) and `.../goldens-dark/` (dark), tracked in git.
- Roborazzi `--update-goldens` mode for development; `--verify` mode for CI.
- The parity gate (Wave A) is extended in Wave D to also validate that goldens were regenerated when tokens.css changes — preventing stale-golden bluff.

Wave E — Close UI-coverage audit gaps

Depends on: Wave D (the screenshot harness is reused for the new Challenges). Can start independently using existing `ComposeTestRule` assertion patterns.

Files modified:

- `app/src/androidTest/kotlin/lava/app/challenges/C31-35*Test.kt` — rewrite the 5 WEAK `Class.forName` reachable-only Challenges (C31, C32, C33, C34, C35) with actual behavior assertions + screenshot capture (Wave D harness)

New Challenges for GAP screens:

- `app/src/androidTest/kotlin/lava/app/challenges/C-XX-ChallengeAccountScreenTest.kt` — GAP: Account screen had no UI test
- `app/src/androidTest/kotlin/lava/app/challenges/C-XX-ChallengeMainScreenTest.kt` — GAP: Main screen post-login had no UI test
- `app/src/androidTest/kotlin/lava/app/challenges/C-XX-ChallengeSearchInputChipsTest.kt` — WEAK: search_input had reachable-only test; add behavior assertions for chip selection/deselection
- `app/src/androidTest/kotlin/lava/app/challenges/C-XX-ChallengeCredentialsManagerTest.kt` — GAP: credentials_manager had no UI test

---

8. §6.AB discrimination requirements per wave

Per §6.AB.3 (the discrimination test for every Challenge before it is declared "covers the feature"), the implementer MUST perform the mutation-rehearsal protocol on every new Challenge before the wave's code is committed:

Protocol (mirrors Sixth Law clause 2):

1. Write the Challenge with behavior assertion on user-visible state.
2. Run against the broken production code (the mutation from the table in §5). Confirm it FAILS with a clear assertion message.
3. Revert the mutation.
4. Run against the fixed production code. Confirm it PASSES.
5. Record both outcomes in the commit body's Bluff-Audit stamp.

Per-wave mutation classes:

Wave	Challenge	Mutation class	Expected failure signal
A	token parity gate	Remove a CSS token value	`exit 1: Drift detected at token`

Wave	Challenge	Mutation class	Expected failure signal
			`'--lava-color-primary-40'`
B	transition Challenge	Replace `LavaMotionScheme.forward` with `Default`	`AssertionError: expected enterTransition != null but was null`
B	shared-element Challenge	Remove `sharedElement`/`sharedBounds` modifiers	Golden perceptual-diff > threshold: missing shared-element region
B	reduced-motion Challenge	Override scale to 1 at 0	Transition frames captured but should be empty
C	Loading/Empty Challenge	Remove loading branch	Skeleton composable not found on screen
C	chip label Challenge	Revert `displayLabel()`	Chip text matches raw id
C	mDNS name Challenge	Return empty friendly name	Screen shows bare IP:port
C	provider count Challenge	Hardcode count	Label reads `4` instead of dynamic count
C	brand font Challenge	Revert to `Default`	Screenshot golden diff
D	visual-regression golden	Flip a color value	Golden perceptual-diff > threshold
D	transition test	Remove `enterTransition`	Assertion on null transition fails
D	overlap test	Add overlapping composable	Overlap assertion fails
E	rewritten C31-C35	Per-Challenge mutation in KDoc	Per-Challenge assertion message

9. File impact map

Every file that will be created or modified across the 5 waves. File paths are relative to repo root.

Wave A — OpenDesign integration (CREATE + MODIFY)

Create:

```
design/lava-opendesign/tokens.css
design/lava-opendesign/DESIGN.md
design/lava-opendesign/manifest.json
design/lava-opendesign/components.html
core/designsystem/src/main/assets/fonts/<brand-font>.ttf
scripts/generate-opendesign-tokens.sh
scripts/check-opendesign-token-sync.sh
scripts/install-opendesign.sh
docs/opendesign-integration.md
tests/designsystem/test-token-parity.sh
```

Modify:

```
core/designsystem/theme/Colors.kt           ← generated section
replaces literals
core/designsystem/theme/AppTypography.kt     ← brand font +
generated from tokens.css
core/designsystem/theme/AppSpaces.kt         ← generated from
tokens.css
core/designsystem/theme/AppShapes.kt         ← generated from
tokens.css
```

Wave B — Transition system (CREATE + MODIFY)

Create:

```
core/navigation/ui/LavaMotionScheme.kt
core/navigation/ui/LavaSharedElementPresets.kt
core/navigation/ui/ReducedMotionAwareTransition.kt
core/designsystem/src/test/.../transition/LavaMotionSchemeTest.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeSearchInputToResultsTransitionTest.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeSearchResultToTopicSharedElementTest.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeReducedMotionTransitionTest.kt
```

Modify:

```
app/.../navigation/MobileNavigation.kt          ← replace Default
assignments
```

```
feature/search_result/.../SearchResultScreen.kt ← add sharedBounds
feature/topic/.../TopicScreen.kt               ← add sharedElement
feature/provider_config/.../ProviderConfigScreen.kt ← add
sharedElement
```

Wave C — Polish (CREATE + MODIFY)

Create:

```
core/designsystem/component/LoadingSkeleton.kt
core/designsystem/component/EmptyState.kt
tests/designsystem/test-loading-skeleton-contract.kt
tests/designsystem/test-empty-state-contract.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeSearchResultLoadingEmptyTest.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeProviderChipDisplayLabelTest.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeMdnsFriendlyNameTest.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeProviderCountLabelTest.kt
app/src/androidTest/kotlin/lava/app/challenges/C-XX-
ChallengeBrandFontAppliedTest.kt
```

Modify:

```
feature/search_result/.../SearchResultScreen.kt   ← Loading
skeleton + Empty state
feature/search_result/.../SearchResultViewModel.kt ← expose
isLoading state
feature/search_input/.../SearchInputProviderChip.kt ←
displayLabel() wiring
core/data/.../LocalNetworkDiscoveryService.kt       ← friendly-
name resolution
feature/onboarding/.../ProviderCountSection.kt       ← dynamic
count
```

Wave D — Visual-regression test infra (CREATE + MODIFY)

Create:

```
core/designsystem/src/test/.../screenshot/
  DesignSystemScreenshots.kt
  TransitionScreenshots.kt
  ThemeOverlapScreenshots.kt
  PaletteOpenDesignParityTest.kt
  ReducedMotionTest.kt
  goldens/                                    ← golden images
(light)
```

```
  goldens-dark/                                ← golden images
(dark)
```

Modify:

```
scripts/ci.sh                                   ← register visual-
regression suite
scripts/check-opendesign-token-sync.sh          ← golden-
freshness check
```

Wave E — Coverage gap closure (MODIFY + CREATE)

Modify:

```
app/src/androidTest/kotlin/lava/app/challenges/
  C31-ChallengeXxxTest.kt                     ← rewrite with
behavior assertions
  C32-ChallengeXxxTest.kt                     ← rewrite
  C33-ChallengeXxxTest.kt                     ← rewrite
  C34-ChallengeXxxTest.kt                     ← rewrite
  C35-ChallengeXxxTest.kt                     ← rewrite
```

Create:

```
app/src/androidTest/kotlin/lava/app/challenges/
  C-XX-ChallengeAccountScreenTest.kt          ← GAP
  C-XX-ChallengeMainScreenTest.kt             ← GAP
  C-XX-ChallengeSearchInputChipsTest.kt       ← WEAK upgrade
  C-XX-ChallengeCredentialsManagerTest.kt     ← GAP
```

---

10. Honest blockers

10.1 Darwin/arm64 emulator gap (§6.AH / §6.X-debt)

Full on-device Challenge execution (C-XX-* from waves B–E) requires an Android emulator booted INSIDE a podman/docker container managed by the `submodules/containers` submodule. The macOS/arm64 host's podman VM does NOT expose `/dev/kvm` or HVF passthrough, so the containerized-emulator path is blocked.

Status: The §6.X macOS allowance was revoked by §6.AH (added 2026-06-03). Host-direct emulator execution is FORBIDDEN for gate runs. The container/VM path on macOS requires either (a) TCG software emulation support in `submodules/containers/pkg/emulator/` (not yet implemented), or (b) a Linux x86_64 gate host (the primary resolution path).

Impact on this spec:

- **Visual-regression layer (Wave D) runs host-side JVM** — NOT blocked. Roborazzi/Paparazzi are Robolectric-backed and need no emulator. This is a major advantage: the visual-regression suite can catch regressions even without emulator access.
- **Device Challenge execution (waves B–E) is BLOCKED on macOS** unless the operator provisions a Linux x86_64 gate host. Each Challenge's KDoc MUST declare its gating status: `@DeviceGate(required = true, note = "Container/VM requirement per §6.AH; run on Linux x86_64 gate host")`.
- **§6.Z distribute gate is BLOCKED** until the §6.AK coverage-intersection gate is satisfied (per-claim covering Challenge must execute PASS on device).

Honest statement:

> Wave D's visual-regression infrastructure runs on any host with a JDK (including macOS). Device Challenges from waves B, C, and E require a container-booted Android emulator and are executed against the artifact on a Linux x86_64 gate host. The implementer writes the Challenges and confirms they compile; the operator runs them on the gate host before the distribute tag.

10.2 `tokens.css` ↔ Compose mapping complexity

CSS custom properties and Compose `Color`/`Dp`/`Shape` values do not have a 1:1 mapping. The codegen script (`scripts/generate-opendesign-tokens.sh`) must handle:

- CSS `--lava-color-primary-40: #B3261E` → Kotlin `val primary40 = Color(0xFFB3261E)`
- CSS `--lava-space-md: 16px` → Kotlin `val md = 16.dp`
- CSS `--lava-shape-corner-small: 8px` → Kotlin `val small = 8.dp`
- CSS font-family strings → Kotlin `FontFamily` constructor
- Media queries (`prefers-color-scheme: dark`) → Kotlin `isDark` parameter

Mitigation: The parity gate only needs to verify the TOKENS THAT EXIST match. The codegen script starts with the subset that maps cleanly (colors, spacing, shapes, border-radii, elevations) and skips unparseable tokens with a WARNING. The remaining tokens are hand-mapped in a dedicated `// region HandCraftedOverrides` section of the generated file, with a comment linking to the CSS line they derive from.

10.3 Roborazzi may not exist in the Compose BOM or may conflict with existing dependencies

Roborazzi is a third-party library (`io.github.takahirom.roborazzi`). If it conflicts with the existing Compose/Robolectric versions, Wave D may need to:

- Pin a specific compatible version
- Fall back to Paparazzi (which has fewer integration points but is more self-contained)
- Use a custom `takeScreenshot()` helper with `drawToBitmap()` + perceptual-diff

Mitigation: The spec names both options; decision #5 delegates the choice. Wave D's first task is a compile-check with Roborazzi; if it fails, fall back to Paparazzi in the same wave.

---

11. Risks and mitigations

#	Risk	Likelihood	Impact	Mitigation
1	Operator decisions 1–4 delayed past Wave A start	High (decisions need human input)	Blocking	Spec marks STARTUS DRAFT and enumerates decisions clearly. Draft the codegen script with placeholder tokens as a pre-work task so Wave A is partly executable without decisions.
2	`tokens.css` → Compose mapping incomplete for complex tokens	Medium	Codegen generates overridable sections	HandCraftedOverrides region with CSS-line cross-references. Parity gate only checks token set intersection.
3	Roborazzi version conflicts	Low–Medium	Wave D may need Paparazzi fallback	Both tools evaluated; Paparazzi is the fallback (more self-contained).
4	Shared-element transitions degrade scrolling performance on low-end devices	Low	Perceived jank	Shared-element transitions are bounded to 350ms and only fire on navigation (not scrolling). Reduced-motion mode disables them entirely. Test on API 28 emulator in the matrix.
5				

#	Risk	Likelihood	Impact	Mitigation
	New Challenges (waves B–E) do not run on macOS (container gap)	High (blocking)	Operator must run on Linux x86_64 gate host	Documented honestly per §10.1. The Challenges compile and the test APK builds on macOS; only execution is gated.
6	Existing tests are flaky due to animation changes	Medium	False negatives in CI	Add `ComposeTestRule.waitForIdle()` before all assertions that follow navigation, per the existing `C27` fix pattern. Reduced-motion-aware test harness.
7	Brand font increases APK size	Low	~200 KB for a variable `.ttf`	Acceptable. If operator objects, subset the font via `fonttools` to latin + cyrillic (Lava's primary scripts).

---

12. Execution order and parallelism

```
Operator decisions (§6)
        │
        ▼
    Wave A ──────────────────────────────────────► (foundation)
        │                                              │
        ├── Wave B (transitions, after easing info) ◄──┘ (no code
dependency)
        │
        ├── Wave C (polish, depends on A's tokens)
        │
        ├── Wave D (visual-regression, after decision 5)
        │
        └── Wave E (coverage, after D's harness or independently)
```

- Waves B and D can start **immediately after Wave A's first commit** (they depend on the directory structure existing, not on token completeness).
- Wave C depends on Wave A's generated token classes — but chips/mDNS/count fixes do NOT depend on tokens and can start in parallel.
- Wave E depends on Wave D's screenshot harness for the screenshot assertions; the behavior assertions can be written before Wave D lands.
- **Recommended agent dispatch order:**
  1. Agent 1: Wave A (tokens.css + codegen + parity gate + docs)

2. Agent 2: Wave B (transition spec + shared-elements + Challenges), starts after Wave A's first commit
3. Agent 3: Wave C chip/mDNS/count fixes (token-independent items), then Loading/Empty (token-dependent after Wave A)
4. Agents 2 and 3 can run simultaneously with Agent 1 after the initial token infrastructure lands
5. Agent 4: Wave D (Roborazzi setup + golden screenshots), after decision 5
6. Agent 5: Wave E (rewrite C31–C35 + GAP screens), after Wave D's harness or standalone with existing assertions

---

13. Constitutional notes

- **§11.4.162:** This spec IS the compliance plan. Wave A makes Lava formally compliant; Wave D adds the mandated visual-regression tests.
- **§6.R:** No hardcoded values in the codegen output — `tokens.css` is the SSoT, and the parity gate enforces it. The codegen script itself does not hardcode colors/typography/spaces (only CSS-parsing logic).
- **§6.AK:** The per-wave Challenge table in §5 is the coverage-intersection contract. Every wave's CHANGELOG entry MUST be matched by the executed Challenges listed there. The implementer MUST run the RED-then-GREEN protocol before claiming a wave is complete.
- **§6.AB:** Every new Challenge carries the falsifiability rehearsal KDoc block. Every transition/shared-element mutation must be tested with the mutation-rehearsal protocol.
- **§6.AH / §6.X-debt:** Device Challenges are container-bound per §6.AH. On macOS they are honestly blocked; on Linux x86_64 they run. The visual-regression layer (Wave D) is JVM-native and runs everywhere. Honest gating per §10.1.
- **§6.Z / §6.AA:** Shipping these changes to users requires the §6.Z pre-distribute device gate AND the §6.AA two-stage distribute. Until the §6.AK coverage-intersection gate is closed, the operator manually verifies per-claim coverage at distribute time.
- **§6.P:** Every distribute of this work MUST have a CHANGELOG entry listing the per-wave user-visible changes AND citing the covering Challenge test names. Version bump per §6.Y applies.
- **§6.W:** No new remotes needed. The `tokens.css` file is committed to this repo (GitHub + GitLab); the OpenDesign MCP server is a development tool, not a new remote.
- **§6.U:** No `sudo`/`su`. The parity gate and codegen script run with user-level permissions. Font assets are placed in `core/designsystem/src/main/assets/fonts/` (standard Android resource path).